

Game of Games Group 3: Ben, Sean, Abby, Juan

Table of Contents:

- Planning and Progress documentation: Page 2
- Code Design Documentation: Page 4
- Deployment Documentation: Page 37
- Responsibilities Documentation: Page 39
- Testing Documentation: Page 45

Planning and Progress – Task Assignments

Sean Balbale's tasks

- Initial setup of Github, Jira, and Slack communication (30 minutes total)
- Documentation writing (2.5 hours total)
 - o Responsibilities document
 - o Deployment document
 - o UML Diagrams (drafts for all classes on github)
 - o Data configuration table
 - o GitHub README.md
- Coded PlayGames.java (45 min), CoinFlip.java (35 min)
 - o Assisted in implementation of GuessTheNumber.java, EvenAndOdd.java, and FindTheThimble.java with Juan and Abby, respectively.
- Ran black-box tests for Coin Flip and Overall Game of Games (30 min total)
- Wrote and executed unit tests GetInputTest.java (1.5 hrs), PlayGames.java (2 hrs), EvenAndOddTest.java (1.5 hrs) and CoinFlip.java (1.5 hrs)
 - o Assisted in implementing unit tests for FindTheThimble and GuessTheNumber

Ben Lyons' tasks

- Coded GetInput.java (50 min) and FindTheRedThread.java (35 min)
- Ran black-box tests for Find The Red Thread (15 min)
- Wrote and executed FindTheRedThreadTest.java (2 hrs)
- Compile meeting notes and documentation for final submission

Abby Gomes' tasks

- Coded FindTheThimble.java (50 min) with Sean Balbale
- Ran black-box tests for Find the Thimble and Guess the Number (45 min total)
- Co-authored unit tests in FindTheThimbleTest.java (2 hrs)
- Create Gantt chart for final documentation submission

Juan Marcano

- Coded EvenAndOdd.java (45 min) and GuessTheNumber.java (45 min) with Sean Balbale
- Assisted in writing of EvenAndOddTest.java with Sean Balbale

Gantt Chart

[illegible]

Code Design – See <https://github.com/sbalbale/Game-Of-Games/blob/main/CodeDesign.md>

GetInput

```
@startuml
class GetInput {
    - scanner: Scanner

    + getInt(prompt: String): int
    + getIntInRange(prompt: String, min: int, max: int, errorMsg:
String): int
    + getOddInt(prompt: String, errorMsg: String): int
    + getChar(prompt: String, validChars: char[], errorMsg: String):
char
}
@enduml
```

Method Glossary

public int **getInt** (String)

- **param1:** String, accepting the prompt to display to the user.
- **return:** integer, representing a valid numeric input.
- **purpose:** Prompts the user for an integer, catches exceptions to prevent crashes, and returns the valid integer.

public int **getIntInRange** (String, int, int, String)

- **param1:** String, prompt to display to the user.
- **param2:** integer, minimum acceptable value.
- **param3:** integer, maximum acceptable value.
- **param4:** String, error message for invalid input.
- **return:** integer, validated number within bounds.
- **purpose:** Prompts the user for a number, looping until the input falls inclusively between the min and max parameters.

public int **getOddInt** (String, String)

- **param1:** String, prompt to display to the user.
- **param2:** String, error message.
- **return:** integer, validated odd number.

- **purpose:** Accepts an integer, verifying it is odd, looping if the user provides an even number.

`public char getChar (String, char[], String)`

- **param1:** String, prompt to display to the user.
- **param2:** char array, specific valid characters allowed.
- **param3:** String, error message.
- **return:** char, validated user selection.
- **purpose:** Prompts the user, reads the character, and validates it against the provided array.

Data Configuration Table

Value & Variable	Input Format	Output Type/Format	Internal Representation
Valid character set, validChars	N/A	Displayed in prompt (e.g., "[H, T]")	char[]

Unit Tests

Class/Method	Purpose	Test Steps	Expected Result	Actual Result	Passed	Failed
GetInput/getInt	Returns valid integer when input is numeric	Call method; enter "5"	Returns 5	Returns 5	X	
GetInput/getInt	Re-prompts on non-numeric input	Call method; enter "abc", then "3"	Re-prompts; returns 3	Re-prompts; returns 3	X	

GetInput/getIntInRange	Returns integer within specified range	Call with min=1, max=6; enter "4"	Returns 4	Returns 4	X	
GetInput/getIntInRange	Rejects integer below range	Call with min=1, max=6; enter "0", then "2"	Re-prompt s; returns 2	Re-prompt s; returns 2	X	
GetInput/getIntInRange	Rejects integer above range	Call with min=1, max=6; enter "9", then "3"	Re-prompt s; returns 3	Re-prompt s; returns 3	X	
GetInput/getIntInRange	Rejects non-numeric input	Call with min=1, max=6; enter "abc", then "2"	Re-prompt s; returns 2	Re-prompt s; returns 2	X	
GetInput/getOddInt	Returns valid odd integer	Call method; enter "5"	Returns 5	Returns 5	X	
GetInput/getOddInt	Rejects even integer	Call method; enter "4", then "3"	Re-prompt s; returns 3	Re-prompt s; returns 3	X	
GetInput/getOddInt	Rejects non-numeric input	Call method; enter "abc", then "7"	Re-prompt s; returns 7	Re-prompt s; returns 7	X	

GetInput/getChar	Returns valid character from set	Call with validChars=['H','T']; enter "H"	Returns 'H'	Returns 'H'	X	
GetInput/getChar	Rejects character not in valid set	Call with validChars=['H','T']; enter "W", then "T"	Re-prompt s; returns 'T'	Re-prompt s; returns 'T'	X	
GetInput/getChar	Rejects lowercase when set is uppercase	Call with validChars=['H','T']; enter "h", then "H"	Re-prompt s; returns 'H'	Re-prompt s; returns 'H'	X	

PlayGames

```

@startuml
class PlayGames {
    - userOverallWins: int
    - compOverallWins: int

    + main(args: String[]): void
    + displayWelcomeMessage(): void
    + displayMainMenu(): int
    + routeToGame(choice: int): void
    + updateOverallScoreboard(userWon: boolean): void
    + displayFinalTally(): void
    + declareOverallWinner(): void
}
@enduml

```

Method Glossary

public static void **main** (String[])

- **param1:** String array, command line arguments.
- **return:** void
- **purpose:** The primary entry point of the program. Checks for the `--test` flag, drives the main game selection loop, and triggers the final tally on exit.

public void **displayWelcomeMessage** ()

- **return:** void
- **purpose:** Prints the initial "Game of Games" welcome banner to the console.

public int **displayMainMenu** ()

- **return:** integer, menu choice between 1 and 6.
- **purpose:** Displays the 1–6 menu options and returns the user's validated selection.

public void **routeToGame** (int)

- **param1:** integer, representing the menu selection.
- **return:** void
- **purpose:** Routes the user to the correct mini-game class based on their menu choice.

public void **updateOverallScoreboard** (boolean)

- **param1:** boolean, indicating if the user won the preceding game.
- **return:** void
- **purpose:** Increments the overall session score and prints the updated tally.

public void **displayFinalTally** ()

- **return:** void
- **purpose:** Displays the final accumulated scores upon quitting.

public void **declareOverallWinner** ()

- **return:** void
- **purpose:** Evaluates the final scores and prints the ultimate winner of the session.

Data Configuration Table

Value & Variable	Input Format	Output Type/Format	Internal Representation
Main menu selection, choice	Integer (1–6)	N/A	int
Overall session scores, userOverallWins , compOverallWins	N/A	Integer values shown on final tally	int

Unit Tests

Class/Method	Purpose	Test Steps	Expected Result	Actual Result	Pas sed	Fai led
PlayGames/routeToGame	Routes choice 1 to GuessThe Number	Call routeToGame(1)	GuessThe Number game initiates	GuessThe Number game initiates	X	
PlayGames/routeToGame	Routes choice 4 to FindTheThimble	Call routeToGame(4)	FindTheThimble game initiates	FindTheThimble game initiates	X	
PlayGames/displayMainMenu	Accepts valid menu selection	Enter "1" at menu prompt	Returns 1	Returns 1	X	
PlayGames/displayMainMenu	Rejects out-of-range numeric input	Enter "9" at menu prompt	Error message displayed; re-prompted	Error message displayed; re-prompted	X	

PlayGames/displayMainMenu	Rejects non-numeric input	Enter "W" at menu prompt	Error message displayed; re-prompted	Error message displayed; re-prompted	X	
PlayGames/main	Input 6 exits loop and calls displayFinalTally	Enter "6" at menu	System exits menu; proceeds to Final Tally	System exits menu; proceeds to Final Tally	X	
PlayGames/displayFinalTally	Displays total wins and losses for both players	After quitting with recorded wins/losses; call displayFinalTally	Total wins and losses displayed for User and Computer	Total wins and losses displayed for User and Computer	X	
PlayGames/declareOverallWinner	Declares user as winner when user has more wins	Set userOverallWins > compOverallWins; call declareOverallWinner	Message naming User as "Winner of the Game of Games"	Message naming User as "Winner of the Game of Games"	X	
PlayGames/declareOverallWinner	Declares computer as winner when computer has more wins	Set compOverallWins > userOverallWins; call declareOverallWinner	Message naming Computer as "Winner of the Game of Games"	Message naming Computer as "Winner of the Game of Games"	X	

PlayGames/declare OverallWinner	Declares draw when wins are equal	Set userOverall Wins == compOverall Wins; call declareOverallWinner	"The Game of Games ends in a draw!" displayed	"The Game of Games ends in a draw!" displayed	X	
---------------------------------	-----------------------------------	---	---	---	---	--

FindTheThimble

```

@startuml
class FindTheThimble {
    - userScore: int
    - compScore: int
    - winThreshold: int

    + playGame(): boolean
    - getBestOutOfValue(): int
    - calculateWinThreshold(bestOutOf: int): void
    - hideThimble(): char
    - getPlayerGuess(): char
    - checkRoundWinner(guess: char, hidden: char): void
    - updateRoundScore(userWonRound: boolean): void
    - checkGameWinner(): boolean
    - declareGameWinner(): void
}
@enduml

```

Method Glossary

public boolean **playGame** ()

- **return:** boolean, returning true if the user wins the game, false if the computer wins.
- **purpose:** Controls the main execution loop for the mini-game.

private int **getBestOutOfValue** ()

- **return:** integer, the best-out-of value.
- **purpose:** Prompts the user for the odd number of rounds to play.

private void **calculateWinThreshold** (int)

- **param1:** integer, the best-out-of odd value.
- **return:** void
- **purpose:** Calculates the required wins to end the game: $(\text{bestOutOf} + 1) / 2$.

private char **hideThimble** ()

- **return:** char, representing 'L' or 'R'.
- **purpose:** Randomly selects the hand the thimble is hidden in.

private char **getPlayerGuess** ()

- **return:** char, representing 'L' or 'R'.
- **purpose:** Prompts the user to guess a hand.

private void **checkRoundWinner** (char, char)

- **param1:** char, the user's guess.
- **param2:** char, the actual hidden location.
- **return:** void
- **purpose:** Compares the parameters, outputs the result, and calls `updateRoundScore`.

private void **updateRoundScore** (boolean)

- **param1:** boolean, true if the user won the round.
- **return:** void
- **purpose:** Increments the appropriate round score.

private boolean **checkGameWinner** ()

- **return:** boolean
- **purpose:** Checks if either player has reached the `winThreshold`.

private void **declareGameWinner** ()

- **return:** void
- **purpose:** Prints the final winner of the mini-game.

Data Configuration Table

Value & Variable	Input Format	Output Type/Format	Internal Representation
Best-out-of rounds, bestOutOf	Odd integer	N/A	int
User hand guess, guess	Character 'L' or 'R'	N/A	char
Hidden thimble location, hidden	N/A	Revealed in text ("Right" / "Left")	char ('L' or 'R')
Win threshold limit, winThreshold	N/A	N/A	int (calculated as (bestOutOf + 1) / 2)
Round score tracker, userScore, compScore	N/A	String ("You: [x], Computer: [y]")	int

Unit Tests

Class/Method	Purpose	Test Steps	Expected Result	Actual Result	Passed	Failed
FindTheThimble/getBestOutOfValue	Accepts valid odd best-of value; threshold calculated correctly	Enter "5"	Win threshold set to 3; round 1 begins	Win threshold set to 3; round 1 begins	X	

FindTheThimble/getBestOutOfValue	Rejects even input	Enter "4"	"Invalid entry. The value must be an odd whole number. Please try again:" Re-prompted	"Invalid entry. The value must be an odd whole number. Please try again:" Re-prompted	X	
FindTheThimble/getBestOutOfValue	Rejects non-numeric input	Enter "abc"	"Invalid entry. The value must be an odd whole number. Please try again:" Re-prompted	"Invalid entry. The value must be an odd whole number. Please try again:" Re-prompted	X	
FindTheThimble/getPlayerGuess	Rejects invalid guess character	Enter "X"	"Invalid input. Please enter L for left or R for right:" Re-	"Invalid input. Please enter L for left or R for right:" Re-	X	

			prompted	prompted		
FindTheThimble/getPlayerGuess	Rejects lowercase input	Enter "l"	"Invalid input. Please enter L for left or R for right:" Re-prompted	"Invalid input. Please enter L for left or R for right:" Re-prompted	X	
FindTheThimble/checkRound Winner	User wins round on correct right-hand guess	Force thimble to 'R'; enter "R"	"Correct ! You win this round." Score: You 1, Computer 0	"Correct ! You win this round." Score: You 1, Computer 0	X	
FindTheThimble/checkRound Winner	User wins round on correct left-hand guess	Force thimble to 'L'; enter "L"	"Correct ! You win this round." Score updated	"Correct ! You win this round." Score updated	X	
FindTheThimble/checkRound Winner	Computer wins round on incorrect guess	Force thimble to 'R'; enter "L"	"Wrong! The computer wins this round." Score:	"Wrong! The computer wins this round." Score:	X	

			You 0, Comput er 1	You 0, Comput er 1		
FindTheThimble/declareGameWinner	Declare s user winner on reachin g thresho ld	Force user to win 3 rounds in best- of-5	"You win Find the Thimble !" Game scorebo ard updated	"You win Find the Thimble !" Game scorebo ard updated	X	
FindTheThimble/declareGameWinner	Declare s comput er winner on reachin g thresho ld	Allow comput er to win 3 rounds in best- of-5	"The compute r wins Find the Thimble !" Game scorebo ard updated	"The compute r wins Find the Thimble !" Game scorebo ard updated	X	

CoinFlip

```
@startuml
class CoinFlip {
    - userScore: int
    - compScore: int
    - winThreshold: int

    + playGame(): boolean
    - getBestOutOfValue(): int
    - calculateWinThreshold(bestOutOf: int): void
    - getPlayerCall(): char
    - flipCoin(): char
    - checkRoundWinner(call: char, result: char): void
}
```

```
- updateRoundScore(userWonRound: boolean): void
- checkGameWinner(): boolean
- declareGameWinner(): void
}
@enduml
```

Method Glossary

public boolean **playGame** ()

- **return:** boolean, returning true if the user wins the game.
- **purpose:** Controls the execution loop for Coin Flip.

private int **getBestOutOfValue** ()

- **return:** integer, the best-out-of value.
- **purpose:** Prompts the user for the odd number of rounds.

private void **calculateWinThreshold** (int)

- **param1:** integer, the best-out-of odd value.
- **return:** void
- **purpose:** Calculates the required wins: $(\text{bestOutOf} + 1) / 2$.

private char **getPlayerCall** ()

- **return:** char, representing 'H' or 'T'.
- **purpose:** Prompts the user to call heads or tails.

private char **flipCoin** ()

- **return:** char, representing 'H' or 'T'.
- **purpose:** Randomly generates the coin flip result.

private void **checkRoundWinner** (char, char)

- **param1:** char, the user's call.
- **param2:** char, the flip result.
- **return:** void

- **purpose:** Compares the call to the flip, outputs the result, and calls `updateRoundScore`.

private void **updateRoundScore** (boolean)

- **param1:** boolean, true if the user won the round.
- **return:** void
- **purpose:** Increments the score.

private boolean **checkGameWinner** ()

- **return:** boolean
- **purpose:** Evaluates if the `winThreshold` is met.

private void **declareGameWinner** ()

- **return:** void
- **purpose:** Prints the game winner.

Data Configuration Table

Value & Variable	Input Format	Output Type/Format	Internal Representation
Best-out-of rounds, bestOutOf	Odd integer	N/A	int
User coin call, call	Character 'H' or 'T'	N/A	char
Coin flip result, result	N/A	String (e.g., "The coin landed on: Heads.")	char ('H' or 'T')
Win threshold limit, winThreshold	N/A	N/A	int (calculated as $(\text{bestOutOf} + 1) / 2$)
Round score tracker, userScore , compScore	N/A	String ("You: [x], Computer: [y]")	int

Unit Tests

Class/Method	Purpose	Test Steps	Expected Result	Actual Result	Passed	Failed
CoinFlip/getBestOutOfValue	Accepts valid odd best-of value; threshold calculated correctly	Enter "7"	Win threshold set to 4; round 1 begins; user prompted for H or T	Win threshold set to 4; round 1 begins; user prompted for H or T	X	
CoinFlip/getBestOutOfValue	Rejects even input	Enter "6"	"Invalid entry. The value must be an odd whole number. Please try again:" Re-prompted	"Invalid entry. The value must be an odd whole number. Please try again:" Re-prompted	X	
CoinFlip/getBestOutOfValue	Rejects non-integer input	Enter "3.5"	"Invalid entry. The value must be an odd whole number. Please try again:"	"Invalid entry. The value must be an odd whole number. Please try again:"	X	

			Re-prompted	Re-prompted		
CoinFlip/getPlayerCall	Rejects invalid call character	Enter "W"	"Invalid input. Please enter H for heads or T for tails:" Re-prompted	"Invalid input. Please enter H for heads or T for tails:" Re-prompted	X	
CoinFlip/getPlayerCall	Rejects lowercase input	Enter "h"	"Invalid input. Please enter H for heads or T for tails:" Re-prompted	"Invalid input. Please enter H for heads or T for tails:" Re-prompted	X	
CoinFlip/checkRoundWinner	User wins round on correct heads call	Force coin to 'H'; enter "H"	"Correct call! You win this round." Score updated	"Correct call! You win this round." Score updated	X	
CoinFlip/checkRoundWinner	User wins round on correct tails call	Force coin to 'T'; enter "T"	"Correct call! You win this round." Score updated	"Correct call! You win this round." Score updated	X	
CoinFlip/checkRoundWinner	Computer	Force coin to 'T'; enter "H"	"Wrong call! The	"Wrong call! The	X	

	wins round on incorrect call		computer wins this round." Score updated	computer wins this round." Score updated		
CoinFlip/flipCoin	Coin result displayed before round winner is announced	Force coin to 'H'; call flipCoin then checkRound Winner	"The coin landed on: Heads." shown before result announcement	"The coin landed on: Heads." shown before result announcement	X	
CoinFlip/declareGameWinner	Declare user winner on reaching threshold	Force user to win 4 rounds in best-of-7	"You win Coin Flip!" Scoreboard updated	"You win Coin Flip!" Scoreboard updated	X	

GuessTheNumber

@startuml

```

class GuessTheNumber {
    - targetNumber: int
    - maxGuesses: int
    - currentGuessesLeft: int
    - numberRange: int

    + playGame(): boolean
    - getRange(): int
    - getMaxGuesses(range: int): int

```

```
- generateTargetNumber(range: int): void
- getPlayerGuess(): int
- evaluateGuess(guess: int): boolean
- declareRoundResult(isCorrect: boolean): void
}
@enduml
```

Method Glossary

public boolean **playGame** ()

- **return:** boolean, returning true if the user wins.
- **purpose:** Controls the execution loop for Guess the Number.

private int **getRange** ()

- **return:** integer, the upper bound of the guessing range.
- **purpose:** Prompts the user to establish the upper bound of the number range.

private int **getMaxGuesses** (int)

- **param1:** integer, the established range.
- **return:** integer, the maximum number of guesses allowed.
- **purpose:** Prompts the user for their guess limit, validating it is no more than half the range.

private void **generateTargetNumber** (int)

- **param1:** integer, the established range.
- **return:** void
- **purpose:** Randomly selects the target number within the range.

private int **getPlayerGuess** ()

- **return:** integer, the user's guess.
- **purpose:** Prompts the user for their current guess.

private boolean **evaluateGuess** (int)

- **param1:** integer, the user's guess.
- **return:** boolean, true if the guess matches the target.

- **purpose:** Compares the guess to the target, returning true if matched, and decrements `currentGuessesLeft` if wrong.

private void **declareRoundResult** (boolean)

- **param1:** boolean, indicating a correct guess.
- **return:** void
- **purpose:** Prints the win/loss state of the game based on the evaluation.

Data Configuration Table

Value & Variable	Input Format	Output Type/Format	Internal Representation
Game range bound, numberRange	Integer	N/A	int
Maximum guesses allowed, maxGuesses	Integer ($\leq$ half of range)	N/A	int
Target number, targetNumber	N/A	N/A	int
User's attempt, guess	Integer	N/A	int
Guesses remaining, currentGuessesLeft	N/A	String (e.g., "Guesses left: 4")	int

Unit Tests

Class/Method	Purpose	Test Steps	Expected Result	Actual Result	Passed	Failed
GuessTheNumber/getRange	Rejects non-integer	Enter "Ten"	"Invalid range. Please	"Invalid range. Please	X	

	range input		try again: " Re-prompted	try again: " Re-prompted		
GuessTheNumber/getMaxGuesses	Accepts valid range and guess count within half the range	Enter range "10", guesses "5"	Computer randomly generates target; number generated	Computer randomly generates target; number generated	X	
GuessTheNumber/getMaxGuesses	Rejects guess count exceeding half the range	Enter range "10", guesses "7"	"The number of guesses is over half the range. Please type a different number"	"The number of guesses is over half the range. Please type a different number"	X	
GuessTheNumber/getPlayerGuess	Rejects non-integer guess input	Enter "Five"	"Invalid guess. Please try	"Invalid guess. Please try	X	

			again" Re-prom pted	again" Re-prom pted		
GuessTheNumber/evaluateGuess	Decrements guesses remaining on incorrect guess	Force target != guess; call evaluateGuess with guesses left = 5	"Wrong guess! Guesses left: 4"	"Wrong guess! Guesses left: 4"	X	
GuessTheNumber/evaluateGuess	Triggers loss on final incorrect guess	Force target != guess; call evaluateGuess with guesses left = 1	Returns false; loss condition triggered	Returns false; loss condition triggered	X	
GuessTheNumber/declareRoundResult	Prints loss message when guesses exhausted	Call declareRoundResult(false)	"Wrong! The computer wins this round." "	"Wrong! The computer wins this round." "	X	
GuessTheNumber/evaluateGuess	Returns true on correct guess	Force guess == target; call evaluateGuess	Returns true	Returns true	X	
GuessTheNumber/declareRoundResult	Prints win message	Call declareRoundResult(true)	"Correct! You	"Correct! You	X	

	e on correct guess		win this round "	win this round "		
GuessTheNumber/playGame	Declare s user winner at end of session	User wins final round	"You win Guess the Numb er!"	"You win Guess the Numb er!"	X	
GuessTheNumber/playGame	Declare s comput er winner at end of session	Computer wins final round	"The comp uter wins Guess the Numb er!"	"The comp uter wins Guess the Numb er!"	X	

EvenAndOdd

```
@startuml
class EvenAndOdd {
    - userScore: int
    - compScore: int
    - winThreshold: int
    - userIsEven: boolean

    + playGame(): boolean
    - assignRoles(): void
    - getBestOutOfValue(): int
    - calculateWinThreshold(bestOutOf: int): void
    - getPlayerNumber(): int
    - getComputerNumber(): int
    - calculateSum(userNum: int, compNum: int): int
    - determineRoundWinner(sum: int): void
}
```

```
- updateRoundScore(userWonRound: boolean): void
- checkGameWinner(): boolean
- declareGameWinner(): void
}
@enduml
```

Method Glossary

public boolean **playGame** ()

- **return:** boolean, returning true if the user wins.
- **purpose:** Controls the execution loop for Even and Odd.

private void **assignRoles** ()

- **return:** void
- **purpose:** Prompts user for 'E' or 'O' and sets userIsEven accordingly.

private int **getBestOutOfValue** ()

- **return:** integer, the best-out-of value.
- **purpose:** Prompts the user for the best-out-of value.

private void **calculateWinThreshold** (int)

- **param1:** integer, best-out-of value.
- **return:** void
- **purpose:** Sets the required threshold to win.

private int **getPlayerNumber** ()

- **return:** integer, the player's chosen number.
- **purpose:** Prompts the user to pick a number.

private int **getComputerNumber** ()

- **return:** integer, the computer's chosen number.
- **purpose:** Randomly generates the computer's number.

private int **calculateSum** (int, int)

- **param1:** integer, user's number.
- **param2:** integer, computer's number.
- **return:** integer, the combined sum.
- **purpose:** Computes the combined sum of both numbers.

private void **determineRoundWinner** (int)

- **param1:** integer, the calculated sum.
- **return:** void
- **purpose:** Uses modulo to determine if the sum is even or odd, compares against roles, and declares the round winner.

private void **updateRoundScore** (boolean)

- **param1:** boolean, true if user won the round.
- **return:** void
- **purpose:** Increments the scores.

private boolean **checkGameWinner** ()

- **return:** boolean
- **purpose:** Evaluates if the winThreshold is met.

private void **declareGameWinner** ()

- **return:** void
- **purpose:** Prints the game winner.

Data Configuration Table

Value & Variable	Input Format	Output Type/Format	Internal Representation
User role assignment, userIsEven	Character 'E' or 'O'	String ("You are even.")	boolean
Best-out-of rounds, bestOutOf	Odd integer	N/A	int

User selected number, userNum	Integer	Displayed in sum equation	int
Computer generated number, compNum	N/A	Displayed in sum equation	int
Sum of numbers, sum	N/A	String equation (e.g., "2 + 4 = 6 is even")	int
Win threshold limit, winThreshold	N/A	N/A	int (calculated as (bestOutOf + 1) / 2)
Round score tracker, userScore, compScore	N/A	String ("You: [x], Computer: [y]")	int

Unit Tests

Class/Method	Purpose	Test Steps	Expected Result	Actual Result	Passed	Failed
EvenAndOdd/assignRoles	Accepts 'E' and assigns even role to user	Enter "E"	"You are even." Roles assigned ; best-out-of prompt displayed	"You are even." Roles assigned ; best-out-of prompt displayed	X	
EvenAndOdd/assignRoles	Rejects invalid role input	Enter "Evenodd"	"Invalid role. Please try again:" Re-prompted	"Invalid role. Please try again:" Re-prompted	X	

EvenAndOdd/getBestOutOfValue	Accepts valid odd best-of value; threshold calculated correctly	Enter "7"	Win threshold set to 4; round 1 begins	Win threshold set to 4; round 1 begins	X	
EvenAndOdd/getBestOutOfValue	Rejects invalid best-of value	Enter "20"	"Invalid value. Please try again:" Re-prompted	"Invalid value. Please try again:" Re-prompted	X	
EvenAndOdd/getPlayerNumber	Rejects invalid number input	Enter "-1"	"Invalid number. Please try again:" Re-prompted	"Invalid number. Please try again:" Re-prompted	X	
EvenAndOdd/determineRoundWinner	Even player wins when sum is even	Set userIsEven=true; call with sum=6 (e.g., 2+4)	"2+4=6 is even, you win this round." Even player score	"2+4=6 is even, you win this round." Even player score	X	

			increme nted	increme nted		
EvenAndOdd/determineRoundWinner	Odd player wins when sum is odd	Set userIsEven= false; call with sum=5 (e.g., 3+2)	"3+2=5 is odd, compute r wins this round." Odd player score increme nted	"3+2=5 is odd, compute r wins this round." Odd player score increme nted	X	
EvenAndOdd/calculateSum	Both numbe rs and sum are displa yed each round	Call calculateSu m(3, compNum); verify output	Both numbers and compute d sum shown with round winner announc ed	Both numbers and compute d sum shown with round winner announc ed	X	
EvenAndOdd/declareGameWinner	Declar es user winne r on reachi ng thresh old	Force user to win enough rounds to meet threshold	"You win Even and Odd!" Scorebo ard updated	"You win Even and Odd!" Scorebo ard updated	X	
EvenAndOdd/declareGameWinner	Declar es compu	Allow computer to win enough	"The compute r wins	"The compute r wins	X	

	ter winne r on reachi ng thresh old	rounds to meet threshold	Even and Odd!" Scorebo ard updated	Even and Odd!" Scorebo ard updated		
--	---	--------------------------------	---	---	--	--

FindTheRedThread

```

@startuml
class FindTheRedThread {
    - TOTAL_SPOOLS: int = 20
    - remainingSpools: int
    - maxPullValue: int
    - redThreadPosition: int
    - isUserTurn: boolean

    + playGame(): boolean
    - getMaxPullValue(): int
    - initializeSpools(): void
    - switchTurn(): void
    - getPlayerPullAmount(): int
    - getComputerPullAmount(): int
    - executePull(amount: int): void
    - checkRedThreadPulled(amount: int): boolean
    - declareGameWinner(userWon: boolean): void
}
@enduml

```

Method Glossary

public boolean **playGame** ()

- **return:** boolean, returning true if the user wins.
- **purpose:** Controls the alternating turn loop for Find the Red Thread.

private int **getMaxPullValue** ()

- **return:** integer, the maximum spools allowed per turn.
- **purpose:** Prompts the user to establish the maximum number of spools allowed to be pulled per turn (1–10).

private void **initializeSpools** ()

- **return:** void
- **purpose:** Resets remainingSpools to 20 and assigns a random location for the redThreadPosition.

private void **switchTurn** ()

- **return:** void
- **purpose:** Toggles the isUserTurn boolean.

private int **getPlayerPullAmount** ()

- **return:** integer, the number of spools to pull.
- **purpose:** Prompts the user to specify how many spools to pull.

private int **getComputerPullAmount** ()

- **return:** integer, the computer's pull amount.
- **purpose:** Calculates or randomly generates the computer's pull amount.

private void **executePull** (int)

- **param1:** integer, the amount of spools pulled.
- **return:** void
- **purpose:** Deducts the pulled amount from remainingSpools.

private boolean **checkRedThreadPulled** (int)

- **param1:** integer, the amount pulled.
- **return:** boolean, true if the pull captured the red thread.
- **purpose:** Checks if the pull action captured the redThreadPosition.

private void **declareGameWinner** (boolean)

- **param1:** boolean, true if the user pulled the thread.

- **return:** void
- **purpose:** Prints the final outcome of the game.

Data Configuration Table

Value & Variable	Input Format	Output Type/Format	Internal Representation
Max spools per turn, maxPullValue	Integer (1-10)	N/A	int
Red thread location, redThreadPosition	N/A	N/A	int
Spools to pull, amount	Integer (1 to maxPullValue)	N/A	int
Remaining spool count, remainingSpools	N/A	Displayed remaining integer	int (initialized at 20)
Turn indicator, isUserTurn	N/A	N/A	boolean

Unit Tests

Class/Method	Purpose	Test Steps	Expected Result	Actual Result	Passed	Failed
FindTheRedThread/getMaxPullValue	Accepts valid max pull value; game initializes with 20 spools	Enter "5"	Game starts with 20 spools; user prompted to pull 1 to 5	Game starts with 20 spools; user prompted to pull 1 to 5	X	

FindTheRedThread/getMaxPullValue	Rejects max pull value greater than 10	Enter "11"	"Invalid entry. The value must be between 1 and 10. Please try again:" Re-prompted	"Invalid entry. The value must be between 1 and 10. Please try again:" Re-prompted	X	
FindTheRedThread/getMaxPullValue	Rejects max pull value of 0	Enter "0"	"Invalid entry. The value must be between 1 and [Max pull value]. Please try again:" Re-prompted	"Invalid entry. The value must be between 1 and [Max pull value]. Please try again:" Re-prompted	X	
FindTheRedThread/getPlayerPullAmount	Rejects pull amount	Set max=5; enter "7"	"Invalid input.	"Invalid input.	X	

	t outside allowe d range		Please enter a numbe r betwee n 1 and 5:" Re- promp ted	Please enter a numbe r betwee n 1 and 5:" Re- promp ted		
FindTheRedThread/getPlay erPullAmount	Rejects pull amoun t exceedi ng remain ing spools	Set max=10, remainingSpo ols=3; enter "5"	"Invali d move. You cannot pull more spools than remain :" Re- promp ted	"Invali d move. You cannot pull more spools than remain :" Re- promp ted	X	
FindTheRedThread/getPlay erPullAmount	Rejects non- numeri c pull input	Enter "abc"	Error messag e display ed; re- promp ted for a valid numbe r	Error messag e display ed; re- promp ted for a valid numbe r	X	
FindTheRedThread/checkR edThreadPulled	Detects user pulling	Force redThreadPosi tion within pull range; call	"You pulled the red thread!	"You pulled the red thread!	X	

	the red thread	checkRedThreadPulled	You win!" Scoreboard updated	You win!" Scoreboard updated		
FindTheRedThread/checkRedThreadPulled	Detects computer pulling the red thread	Force redThreadPosition within computer pull range; call checkRedThreadPulled on computer's turn	"The computer pulled the red thread! The computer wins!" Scoreboard updated	"The computer pulled the red thread! The computer wins!" Scoreboard updated	X	
FindTheRedThread/executePull	Remaining spool count decrements correctly after each pull	Set remainingSpools=20; call executePull(3)	17 spools remain displayed; count continues decreasing each turn	17 spools remain displayed; count continues decreasing each turn	X	
FindTheRedThread/switchTurn	Turns alternate correctly	Call switchTurn repeatedly; observe	Game alternates user →	Game alternates user →	X	

	ly between user and computer	isUserTurn value	computer → user until red thread is found	computer → user until red thread is found		
--	------------------------------------	---------------------	--	--	--	--

Unified Data Configuration Table

Class / Method	Value & Variable	Input Format	Output Type/Form at	Internal Representati on
PlayGames / displayMainMenu	Main menu selection, choice	Integer (1–6)	N/A	int
PlayGames / displayFinalTally	Overall session scores, userOverallWins , compOverallWins	N/A	Integer values shown on final tally	int
FindTheThimble / getBestOutOfValue	Best-out-of rounds, bestOutOf	Odd integer	N/A	int
FindTheThimble / getPlayerGuess	User hand guess, guess	Character 'L' or 'R'	N/A	char
FindTheThimble / hideThimble	Hidden thimble location, hidden	N/A	Revealed in text ("Right" / "Left")	char ('L' or 'R')

CoinFlip / getBestOutOfValue	Best-out-of rounds, bestOutOf	Odd integer	N/A	int
CoinFlip / getPlayerCall	User coin call, call	Character 'H' or 'T'	N/A	char
CoinFlip / flipCoin	Coin flip result, result	N/A	String (e.g., "The coin landed on: Heads.")	char ('H' or 'T')
GuessTheNumber / getRange	Game range bound, numberRange	Integer	N/A	int
GuessTheNumber / getMaxGuesses	Maximum guesses allowed, maxGuesses	Integer ($\leq$ half of range)	N/A	int
GuessTheNumber / generateTargetNum ber	Target number, targetNumber	N/A	N/A	int
GuessTheNumber / getPlayerGuess	User's attempt, guess	Integer	N/A	int
GuessTheNumber / evaluateGuess	Guesses remaining, currentGuessesL eft	N/A	String (e.g., "Guesses left: 4")	int
EvenAndOdd / assignRoles	User role assignment, userIsEven	Character 'E' or 'O'	String ("You are even.")	boolean
EvenAndOdd / getBestOutOfValue	Best-out-of rounds, bestOutOf	Odd integer	N/A	int

EvenAndOdd / getPlayerNumber	User selected number, userNum	Integer	Displayed in sum equation	int
EvenAndOdd / getComputerNumber	Computer generated number, compNum	N/A	Displayed in sum equation	int
EvenAndOdd / calculateSum	Sum of numbers, sum	N/A	String equation (e.g., "2 + 4 = 6 is even")	int
FindTheRedThread / getMaxPullValue	Max spools per turn, maxPullValue	Integer (1-10)	N/A	int
FindTheRedThread / initializeSpools	Red thread location, redThreadPosition	N/A	N/A	int
FindTheRedThread / getPlayerPullAmount	Spools to pull, amount	Integer (1 to maxPullValue)	N/A	int
FindTheRedThread / executePull	Remaining spool count, remainingSpools	N/A	Displayed remaining integer	int (initialized at 20)
FindTheRedThread / switchTurn	Turn indicator, isUserTurn	N/A	N/A	boolean
Best-out-of Mini-Games / calculateWinThreshold	Win threshold limit, winThreshold	N/A	N/A	int (calculated as (bestOutOf + 1) / 2)

Best-out-of Mini-Games / updateRoundScore	Round score tracker, userScore , compScore	N/A	String ("You: [x], Computer: [y]")	int
--	--	-----	--	-----

Game of Games Deployment Document

1. Access and Download

The source code for this project is hosted on GitHub. You can obtain it by cloning the repository to your local machine. Open your terminal and run the following commands:

```
git clone [https://github.com/sbalbale/Game-Of-Games](https://github.com/sbalbale/Game-Of-Games)
cd Game-Of-Games
```

2. Configuration & Prerequisites

Before compiling and running the game, ensure your host system meets the following software requirements:

- **Java Development Kit (JDK):** Version 23 or higher installed and configured in your system's PATH.
- **Apache Maven:** Version 3.6 or higher installed to manage dependencies, compile the project, and execute the runtime variations.

3. Compilation

The project utilizes Apache Maven for build lifecycle management. To compile the Java source files, execute the following command from the root directory of the repository (where the `pom.xml` file is located):

```
mvn clean compile
```

This command cleans any previous builds and compiles the latest `.java` files into executable `.class` binaries.

4. Execution Procedures

The application supports two distinct execution variations. Use the corresponding Maven commands to launch your desired mode.

Variation A: Player Mode (Standard Execution)

This is the standard mode intended for end-users. No hidden variables or internal states are exposed during normal gameplay.

```
mvn exec:java
```

Variation B: Test Mode (Debugging & Cheating)

This mode is strictly for testing and debugging. It exposes hidden internal states (such as the location of the thimble, the computer's chosen number, or the exact position of the red thread) to allow QA testers to verify the game logic without relying on random chance.

To run the game in Test Mode, append the `--test` argument:

```
mvn exec:java -Dexec.args="--test"
```

(Note: You will see a `[SYSTEM] Executing in Test Mode...` banner upon successful launch, and cheat outputs will be prefixed with `[TEST MODE]` during gameplay).

5. File Directory Definitions

For a complete glossary defining all project files, class architectures, and configuration dependencies, please refer to the `README.md` file located at the root of the GitHub repository.

Responsibilities Document

Project: Game of Games (Implementation Phase)

Course: Trinity College

Due: April 29, 2026 — 5:00 p.m.

Document prepared by: Sean Balbale

Last updated: April 27, 2026

Team Members

Name	GitHub
Sean Balbale	sbalbale
Ben Lyons	belyons123
Abigail Gomes	abeeinthesky
Juan Marcano	juandmarcanoc

Responsibility Breakdown

Sean Balbale

Project Infrastructure & Setup

- Created and configured the GitHub repository (GOG-4)
- Configured the Kanban board and Gantt chart in Jira (GOG-6)
- Established and managed the Slack communication channel (GOG-7)

Documentation

- Drafted all class UML diagrams, including `.puml` source and rendered `.svg` exports for all six classes (GOG-8)
- Created Method Glossaries for all classes (GOG-14)
- Built the unified Data Configuration Table across all classes (GOG-15)
- Designed the Player Mode and Test Mode architecture (GOG-18)
- Wrote the Deployment Document (`Deployment.md`) (GOG-25)
- Authored this Responsibilities Document (GOG-39)
- Updating the GitHub `README.md` with full file directory (GOG-43 — in progress)

Coding

- Implemented `PlayGames.java` — main menu loop, game routing switch, injectable `BooleanSupplier` runners for testability, overall scoreboard, final tally, and overall winner declaration (GOG-16)
- Added `--test` flag detection and `isTestMode` static flag architecture to `PlayGames.java` (GOG-24)
- Implemented `CoinFlip.java` — full best-of-N game loop, win threshold calculation, random coin flip, round winner evaluation, test-mode reveal, score display (GOG-20)
- Co-authored `GuessTheNumber.java` with Juan Marcano — number-of-rounds loop, range and guess-limit prompts, random target generation, guess evaluation with hint messages, test-mode reveal (GOG-21)
- Co-authored `EvenAndOdd.java` with Juan Marcano — role assignment, best-of-N threshold, round play, sum parity evaluation, score tracking (GOG-22)
- Co-authored `FindTheThimble.java` with Abigail Gomes — random thimble placement, L/R guess validation, round winner evaluation, test-mode reveal (GOG-19)

Testing

- Ran and recorded all black-box tests for Coin Flip, sunny and rainy day paths (GOG-27)
- Ran and recorded all black-box tests for the Overall Game of Games, sunny and rainy day paths (GOG-31)
- Authored and executed all unit tests for GetInput (GetInputTest.java) (GOG-32)
- Authored and executed all unit tests for PlayGames (PlayGamesTest.java) (GOG-33)
- Authored and executed all unit tests for CoinFlip (CoinFlipTest.java) (GOG-35)
- Co-authored unit tests for GuessTheNumber (GuessTheNumberTest.java) with Juan Marcano (GOG-36)
- Authored and executed all unit tests for EvenAndOdd (EvenAndOddTest.java) (GOG-37)
- Co-authored unit tests for FindTheThimble (FindTheThimbleTest.java) with Abigail Gomes (GOG-34)

Ben Lyons

Coding

- Implemented GetInput.java — all validated input methods (getInt, getIntInRange, getOddInt, getChar), resilient to non-numeric and out-of-range input (GOG-17)
- Implemented FindTheRedThread.java — 20-spool game loop, random red thread placement, alternating user/computer turns, pull validation, remaining spool display, test-mode reveal, win detection (GOG-23)

Testing

- Ran and recorded all black-box tests for Find the Red Thread, sunny and rainy day paths (GOG-30)
- Authored and executed all unit tests for FindTheRedThread (FindTheRedThreadTest.java) (GOG-38)

Abigail Gomes

Coding

- Co-authored `FindTheThimble.java` with Sean Balbale — random thimble placement, L/R guess validation via `GetInput`, round winner comparison and messaging, best-of-N threshold, score tracking, test-mode integration (GOG-19)

Testing

- Running black-box tests for Find the Thimble, sunny and rainy day paths (GOG-26 — in progress)
- Running black-box tests for Guess the Number, sunny and rainy day paths (GOG-28 — in progress)
- Co-authoring unit tests for `FindTheThimble` (`FindTheThimbleTest.java`) with Sean Balbale (GOG-34 — in review)

Juan Marcano

Coding

- Co-authored `EvenAndOdd.java` with Sean Balbale — role assignment (E/O), best-of-N best-out-of prompt, round play loop, sum parity evaluation, score tracking, game winner declaration (GOG-22)
- Co-authored `GuessTheNumber.java` with Sean Balbale — multi-round outer loop, range and guess-limit prompts, random target generation, guess evaluation, loss messaging (GOG-21)

Testing

- Authored `EvenAndOddTest.java` — unit tests for threshold calculation, score updating, role-based round winner determination, and game winner declaration

Summary Table

Task	Sean	Ben	Abigail	Juan
------	------	-----	---------	------

GitHub setup	✓			
Kanban / Gantt	✓			
Slack channel	✓			
UML diagrams (all classes)	✓			
Method glossaries	✓			
Data configuration table	✓			
Deployment document	✓			
Responsibilities document	✓			
README	✓			
PlayGames.java	✓			
GetInput.java		✓		
CoinFlip.java	✓			
GuessTheNumber.java	✓ (lead)			✓ (co)
EvenAndOdd.java	✓ (lead)			✓ (co)
FindTheThimble.java	✓ (co)		✓ (lead)	
FindTheRedThread.java		✓		
Black-box — Coin Flip	✓			

Black-box — Guess the Number			✓	
Black-box — Even and Odd	✓			
Black-box — Find the Thimble			✓	
Black-box — Find the Red Thread		✓		
Black-box — Overall Game of Games	✓			
Unit tests — GetInput	✓			
Unit tests — PlayGames	✓			
Unit tests — CoinFlip	✓			
Unit tests — GuessTheNumber	✓ (co)			✓ (co)
Unit tests — EvenAndOdd	✓ (lead)			✓ (co)
Unit tests — FindTheThimble	✓ (co)		✓ (lead)	
Unit tests — FindTheRedThread		✓		

All Jira ticket references (GOG-##) are verifiable at <https://gameofgames.atlassian.net>

Full Testing Documentation

Black-Box Sunny/Rainy day testing for all classes

Find the Red Thread Testing (after testing):

(had to use screenshots as that's all I was provided with)

Sunny Day Paths

EXAMPLE BEFORE TESTING

RUN	NUM	TEST	EXPECTED RESULT	ACTUAL RESULT	CORRECT	INCORRECT
X	1	Find the Red Thread, valid max pull value <u>accepted</u> and game initialized with 20 spools. Input: 5	Game starts with 20 spools; user prompted to pull 1 to 5.	Game starts w/ twenty spools and 5 is max pull val	X	
X	2	Find the Red <u>Thread</u> , user wins when their pulled spools include the red thread. Pull a number of spools that <u>includes</u> the red thread's position.	"You pulled the red thread! You <u>win</u> !" Scoreboard updated; returns to main menu.	"You Win Red Thread!"	X	
X	3	Find the Red <u>Thread</u> , computer wins when its pulled spools include the red thread. Pull spools that miss the red thread; let <u>computer's</u> turn find it.	"The computer pulled the red thread! The computer wins!" Scoreboard updated; returns to main menu.	"Computer wins Red Thread!"	X	
X	4	Find the Red <u>Thread</u> , remaining spool count decreases correctly after each turn. Input: 3	17 spools remain displayed after user pulls 3; count continues decreasing each turn.	"Spools Remaining: 17"	X	
X	5	Find the Red Thread, turns alternate correctly between user and computer. Pull spools that miss the red thread each turn.	Game alternates user → <u>computer → user</u> → computer until red thread is found.	User -> Computer -> User, etc.	X	

Rainy Day Paths						
EXAMPLE BEFORE TESTING						
RUN	NUM	TEST	EXPECTED RESULT	ACTUAL RESULT	CORRECT	INCORRECT
X	1	Find the Red Thread, max pull value greater than 10 rejected. Input: 11	"Invalid entry. The value must be between 1 and 10. Please try again:" Re-prompted.	"Invalid Input: must be an integer between 1 & 10:" Re-prompted	X	
X	2	Find the Red Thread, max pull value of 0 rejected. Input: 0	"Invalid entry. The value must be between 1 and 10. Please try again:" Re-prompted.	"Invalid Input: must be an integer between 1 & 10:" Re-prompted	X	
X	3	Find the Red Thread, pull amount outside the allowed range rejected. Input: Max=5; Pull attempt: 7	"Invalid input. Please enter a number between 1 and 5:" Re-prompted.	"Invalid input: must be an integer between 1 and 5:" re-prompted	X	
X	4	Find the Red Thread, pull amount exceeding remaining spools rejected.	"Invalid move. You cannot pull more spools than remain:" Re-prompted.	Automatically adjusts to pull amount so that there are 0 spools left		X
		Input: Max=10; 3 spools remain; Pull: 5				
X	5	Find the Red Thread, non-numeric pull input rejected. Input: abc	Error message displayed; re-prompted for a valid number.	"Invalid input: must be an integer between 1 & 10: re=prompted"	X	

Note on Rainy Day Test #4: Not necessarily an error, just a feature in the code that skips having to prompt user again and goes straight to pulling the largest amount that would be correct.

CoinFlip Testing:

Sunny Day Paths — After Testing

Run	#	Test	Expected Result	Actual Result	Pass	Fail
X	1	Valid odd best-of value accepted and win threshold set correctly. Input: 7	Win threshold set to 4; round 1 begins; user prompted for H or T.	Threshold = $(7+1)/2 = 4$. Prints "First to 4 wins takes the game." Round 1 begins; user prompted: "Call it — enter H for heads or T for tails:"	X	

X	2	User wins round with correct heads call. Input: H (when coin lands Heads)	"Correct call! You win this round." Score updated.	flipCoin() executes before getPlayerCall(). checkRoundWinner() prints "The coin landed on: Heads." then "Correct call! You win this round." then updated score.	X	
X	3	User wins round with correct tails call. Input: T (when coin lands Tails)	"Correct call! You win this round." Score updated.	Same execution path as test 2. Prints "The coin landed on: Tails." then "Correct call! You win this round." then updated score.	X	
X	4	Computer wins round — wrong call. Input: H (when coin lands Tails)	"Wrong call! The computer wins this round." Score updated.	checkRoundWinner() prints "The coin landed on: Tails." then "Wrong call! The computer wins this round." then updated score.	X	
X	5	Coin result displayed before round winner announced. Input: H	"The coin landed on: Heads." shown before result announcement.	flipCoin() is called before getPlayerCall() in playGame(). checkRoundWinner() prints the coin result first, then the correct/wrong call message. Order confirmed correct.	X	
X	6	User wins game upon reaching the threshold. Win 4 rounds in a best-of-7.	"You win Coin Flip!" Scoreboard updated.	checkGameWinner() returns true when userScore >= 4. declareGameWinner() prints "You win Coin Flip!" playGame() returns true to PlayGames, which updates the overall scoreboard.	X	

Rainy Day Paths — After Testing

Run	#	Test	Expected Result	Actual Result	Pass	Fail
X	1	Even best-of value rejected. Input: 6	"Invalid entry. The value must be an odd whole number.	getOddInt() checks num2 % 2 == 1; 6 fails. Re-prompts with "Invalid input: must be an odd positive integer." Functionally re-prompts		X

			Please try again:" Re-prompted.	correctly; error message wording does not match design spec.		
X	2	Non-integer best-of value rejected. Input: 3.5	"Invalid entry. The value must be an odd whole number. Please try again:" Re-prompted.	"3.5" fails num.matches("\\d+") since the decimal point is not matched by a digit-only regex. Re-prompts with same message as test 1. Functionally correct; error message wording does not match design spec.		X
X	3	Invalid coin call character rejected. Input: W	"Invalid input. Please enter H for heads or T for tails:" Re-prompted.	getChar(new char[]{'H','T'}) — 'W' is not found in the valid char array. Re-prompts with "Error: enter one of these characters: [H, T]" Functionally correct; error message wording does not match design spec.		X
X	4	Lowercase coin call rejected. Input: h	"Invalid input. Please enter H for heads or T for tails:" Re-prompted.	GetInput.getChar() does not convert to uppercase 'h' != 'H', so input is rejected. Re-prompts with same message as test 3.		X

Note: The failing cases are not due to failures of functionality, just not perfectly matching the wording from the expected result.

PlayGames Testing:

Sunny Day Paths — After Testing

Run	#	Test	Expected Result	Actual Result	Pass	Fail
X	1	User selects a valid game. Input: 1	System initiates the "Guess the Number" game use case.	Prints "You selected: Guess the Number", creates GuessTheNumber, calls playGame().	X	

X	2	User selects a valid game. Input: 4	System initiates the "Find the Thimble" game use case.	Prints "You selected: Find the Thimble", creates FindTheThimble, calls playGame().	X	
X	3	User chooses to exit the application. Input: 6	System exits the menu and proceeds to display final tally.	Sets keepPlaying = false, exits the while loop, calls displayFinalTally() then declareOverallWinner().	X	
X	4	Final tally displayed after playing. Input: 6	Computer displays total wins and losses for both user and computer.	Prints "=== Final Tally ===", "Total User Wins: N", "Total Computer Wins: N".	X	
X	5	User achieves more wins than computer, then quits.	System names user as the "Winner of the Game of Games".	Prints "Congratulations! You are the Winner of the Game of Games!"	X	
X	6	Computer achieves more wins than user, then quits.	System names computer as the "Winner of the Game of Games".	Prints "The Computer is the Winner of the Game of Games!"	X	

Rainy Day Paths — After Testing

Run	#	Test	Expected Result	Actual Result	Pass	Fail
X	1	Invalid numeric menu choice. Input: 9	"Invalid choice. Please enter a number between 1 and 6." Re-prompted.	GetInput.getIntInRange() re-prompts with "Invalid input: must be an integer between 1 & 6:". Functionally re-prompts correctly; error message wording does not match design spec.	X	
X	2	Invalid character	"Invalid choice. Please enter a	"W" fails num.matches("\\d+"); same re-prompt message as above.	X	

		menu choice. Input: W	number between 1 and 6." Re- prompted.	Functionally re-prompts correctly; error message wording does not match design spec.		
X	3	Equal wins — declare a draw.	System displays: "The Game of Games ends in a draw!"	Prints "The Game of Games ends in a draw!" exactly.	X	

Note: Error message wording in rainy-day paths is functionally correct but differs from the design spec. GetInput produces generic range/type error messages rather than the spec wording.

Guess the Number — Sunny Day Paths — After Testing						
RUN	NUM	TEST	EXPECTED RESULT	ACTUAL RESULT	PASS	FAIL
X	1	Valid range and guess count accepted when guesses are at most half the range. Input: Range: 10, Guesses : 5	The computer randomly generates a number to guess based on the range. Number: 7	getRange() accepts 10 via getIntInRange(2, MAX_VALUE). maxAllowedGuesses = 10/2 = 5; getMaxGuesses(10) accepts 5 via getIntInRange(1, 5). generateTargetNumber(10) produces a random int in [1, 10]. Game loop begins.	X	
X	2	The user makes an incorrect guess	The number of guesses decreases , indicated	evaluateGuess() returns false; currentGuessesLeft decrements from 5 to 4. declareRoundResult(false) prints: "Wrong guess! Guesses left: 4"	X	

		while still having guesses. Guesses left: 5	by the following message: "Wrong guess! Guesses left: 4"			
X	3	The user makes an incorrect guess while not having guesses left in a round. Guesses left: 1	"Wrong! The computer wins this round."	evaluateGuess() returns false; currentGuessesLeft decrements 1 → 0. declareRoundResult(false) prints "Wrong guess! Guesses left: 0". While loop exits (0 > 0 is false). Prints "Wrong! The computer wins this round!" and returns false.	X	
X	4	User wins round with correct guess within the allowed number of guesses. Enter the correct number within the guess.	"Correct! You win this round" and starts the next run.	evaluateGuess() returns true (no decrement). declareRoundResult(true) prints: "Correct! You have won this round!" playGame() returns true to PlayGames.	X	
X	5	The final round is won by	"You win Guess the Number!"	declareGameResult() runs and displays "You win Guess the Number!" playGame() returns true.	X	

		the player.		PlayGames.updateOverallScoreboard(true) increments userOverallWins and prints the overall scoreboard.		
X	6	The final round is won by the computer.	"The computer wins Guess the Number!"	declareGameResult() runs and displays "The computer wins Guess the Number!" playGame() returns true. playGame() returns false. PlayGames.updateOverallScoreboard(false) increments compOverallWins and prints the overall scoreboard.	X	
Guess the Number — Rainy Day Paths — After Testing						
RUN	NUM	TEST	EXPECTED RESULT	ACTUAL RESULT	PASS	FAIL
X	1	The user types in a non-integer amount for the range. Guess: "Ten"	"Invalid range. Please try again:"	"Ten" fails num.matches("\\d+") in getIntInRange(). Prints: "Invalid input: must be an integer between 2 & 2147483647:" and re-prompts. Functionally correct; error message wording does not match design spec.	X	
X	2	Valid range and guess count is over half of the range. Input: Range: 10, Guesses : 7	"The number of guesses is over half the range. Please type a different number"	maxAllowedGuesses = 10/2 = 5. getIntInRange(1, 5) rejects 7 (7 > 5). Prints: "Invalid input: must be an integer between 1 & 5:" and re-prompts. Functionally correct; error message wording does not match design spec.	X	
X	3	The user types in a non-integer	"Invalid guess. Please try again"	"Five" fails num.matches("\\d+") in getIntInRange(1, range). Prints: "Invalid input: must be an integer between 1 & 10:" and re-prompts. Functionally	X	

		guess. Guess: "Five"		correct; error message wording does not match design spec.		
--	--	----------------------------	--	--	--	--

Even and Odd — Sunny Day Paths — After Testing						
RUN	NUM	TEST	EXPECTED RESULT	ACTUAL RESULT	PASS	FAIL
X	1	Player selects Even role; computer is assigned Odd. Input: E	Roles assigned for the entire game; best-out-of prompt displayed: "You are even."	getChar({'E','O'}) accepts 'E'. userIsEven = true. Prints "You are even." getBestOutOfValue() is then called; getOddInt() prompts for best-out-of value.	X	
X	2	Valid odd best-out-of value accepted and win threshold set correctly. Input: 7	Win threshold set to 4; round 1 begins.	getOddInt() accepts 7 (7 % 2 == 1). calculateWinThreshold(7): winThreshold = (7+1)/2 = 4. Prints "First to 4 wins takes the game." Game loop begins.	X	
X	3	Even player wins round when sum of both numbers is even. Enter an integer which,	Even player's score incremented; result shown e.g. "2+4=6 is even, you win this round."	sum % 2 == 0 and userIsEven = true → sumIsEven == userIsEven → userWonRound = true. userScore++. Prints "[u] + [c] = [sum] is even." then "You win this round!" and running score.	X	

		based on the computer's pick, results in an even sum.				
X	4	Odd player wins round when sum of both numbers is odd. Enter an integer which, based on the computer's pick, results in an odd sum.	Odd player's score incremented; result shown e.g. "3+2=5 is odd, computer wins this round."	sum % 2 == 1 and userIsEven = true → sumIsEven != userIsEven → userWonRound = false. compScore++. Prints "[u] + [c] = [sum] is odd." then "Computer wins this round!" and running score.	X	
X	5	Both players' numbers and their sum are displayed each round. Input: 3	Both numbers and computed sum shown with the round winner announced.	playGame() calls getPlayerNumber() and getComputerNumber(); computes calculateSum() and determines parity. Prints "[userNum] + [compNum] = [sum] is [even/odd]." before the round winner message. Order confirmed correct.	X	
X	6	User wins the game upon reaching	"You win Even and Odd!" Scoreboard updated;	checkGameWinner() returns true when userScore >= winThreshold. declareGameWinner() prints "You win Even and Odd!". playGame() returns true;	X	

		the threshold . Win enough rounds to meet the threshold .	returns to main menu.	PlayGames.updateOverallScoreboard (true) displays overall scoreboard. Menu re-displayed.		
X	7	Computer wins the game upon reaching threshold . Allow computer to win enough rounds.	"The computer wins Even and Odd!" Scoreboard updated; returns to main menu.	checkGameWinner() returns true when compScore >= winThreshold. declareGameWinner() prints "The computer wins Even and Odd!". playGame() returns false; overall scoreboard updated. Menu re-displayed.	X	

Even and Odd — Rainy Day Paths — After Testing

RUN	NUM	TEST	EXPECTED RESULT	ACTUAL RESULT	PASS	FAIL
X	1	Player selects an invalid role. Role: "Evenodd"	"Invalid role. Please try again:"	"Evenodd".length() == 7 ≠ 1; fails length check in getChar(). Prints "Error: enter one of these characters: [E, O]" and re-prompts. Functionally correct; error message wording does not match design spec.	X	
X	2	Best-out-of value and win threshold are invalid. Best: 20, Threshold: -1	"Invalid value. Please try again:"	getOddInt() checks 20 % 2 == 1 → false (20 is even). Prints "Invalid input: must be an odd positive integer:" and re-prompts. Note: a threshold of -1 cannot be entered directly; the design spec describes a combined invalid state, but the code prevents it at input. Functionally correct.	X	
X	3	The player	"Invalid number.	"-1" fails num.matches("\\d+") in getIntInRange(1, 5) (leading minus is	X	

		inputs an invalid integer. Number: -1	Please try again:"	not a digit). Prints "Invalid input: must be an integer between 1 & 5:" and re-prompts. Functionally correct; error message wording does not match design spec.		
--	--	---------------------------------------	--------------------	---	--	--

Find The Thimble Testing:

Sunny Day Paths- After Testing

RUN	NUM	TEST	EXPECTED RESULT	ACTUAL RESULT	CORRECT	INCORRECT
X	1	Valid odd best-out-of value accepted and win threshold set correctly. Input: 5	Win threshold set to 3, round 1 begins.	Program calculates $(5 + 1) / 2 = 3$, prints: "First to 3 wins takes the game."	X	
X	2	The user wins the round by correctly guessing the right hand. Input: R (when thimble is in right hand)	"Correct! You win this round." Score displayed as You: 1, Computer:0.	Program outputs the same: "Correct! You win this round." Score is incremented and displayed as You: 1, Computer: 0.	X	
X	3	The user wins the round by correctly	"Correct! You win this round." Score	Program outputs the same: "Correct! You	X	

		guessing the left hand. Input: L (when thimble is in left hand)	updated accordingly.	win this round." User score is incremented and displayed as You: 1, Computer: 0.		
X	4	The computer wins round when the user guesses Incorrectly. Input: L (when thimble is in right hand)	"Wrong! The computer wins this round." Score displayed as You: 0, Computer: 1.	Program outputs the same: "Wrong! The computer wins this round." Computer score is incremented and displayed as You: 0, Computer: 1.	X	
X	5	The user wins the game upon reaching the win threshold. Win 3 rounds in a best-of-5.	"You win, Find the Thimble!" Game scoreboard updated.	Program outputs the same: "You win, Find the Thimble!" The Game scoreboard is updated.	X	
X	6	The computer wins the game upon reaching the win threshold. Allow	"The computer wins, Find the Thimble!" Game scoreboard updated.	Program outputs the same: "Computer wins, Find the Thimble!" The Game scoreboard is updated.	X	

		the computer to win 3 rounds in a best-of-5.				
--	--	--	--	--	--	--

Rainy Day Paths- After Testing

RUN	NUM	TEST	EXPECTED RESULT	ACTUAL RESULT	CORRECT	INCORRECT
X	1	Even best-out-of value rejected. Input: 4	"Invalid entry. The value must be an odd whole number. Please try again:" Re-prompted.	Input is rejected by getOddInt() with "Invalid entry. The value must be an odd whole number. Please try again:" User is re-prompted until a valid odd number is entered.	X	
X	2	Non-numeric best-out-of value rejected. Input: abc	"Invalid entry. The value must be an odd whole number. Please try again:" Re-prompted.	Input is rejected by getOddInt() with "Invalid entry. The value must be an odd whole number. Please try again:" User is re-prompted until a valid odd number is entered.	X	
X	3	Invalid hand	"Invalid input.	Input is rejected by	X	

		guess character rejected. Input: X	Please enter L for left or R for right:" Re-prompted.	getChar({'L','R'}) with "Invalid input. Please enter L for left or R for right:" User is re-prompted until valid input.		
X	4	Lowercase hand guess rejected. Input: l	"Invalid input. Please enter L for left or R for right:" Re-prompted.	Lowercase 'l' is rejected since only uppercase 'L' or 'R' are accepted; System outputs: "Invalid input. Please enter L for left or R for right:" Re-prompted.	X	